

And of course, all this new business includes a replacement for the *nskintrd* tool, which just happens to be a masterpiece of artistic, uh, coding! Following a modular architecture, "...most of the *initrd* generation functionality in Dracut is provided by a bunch of generator modules that are sourced by the main Dracut script to install specific functionality into the *initrd*."

Yes, all they had to do to introduce Plymouth support was write a new module, and the next time a new ground-busting technology comes up that promises an even more thunder-strikingly brilliant boot experience, all they have to do is write a puny new module to introduce functionality into the *initramfs*. Cool, huh? The Fedora developers are a bright lot, hands down!

KMS and Plymouth

Close to the beginning, all users would see when they booted their system was a gazillion lines of garbled (or seemingly garbled) text—a verbose output, as each and every step of the boot process was completed. While it would 'turn on' the geekily inclined part of the audience, it would scare the majority to death.

Then came a set of revolutionary patches to the kernel, which never made it into the mainline, but nonetheless used VESA BIOS extensions to produce a pretty boot screen at the native screen resolution with a working status bar (as well as giving the verbose output, decorated with pretty overlay graphics, if so requested, by the booter). Compare that to Windows, which until Windows 7, could achieve no more than 640x480 at 16-bit colour.

And pretty recently the buzzword doing the rounds is KMS. In simple terms, in KMS (Kernel Modesetting), setting the screen resolution and colour depth as well as the video memory is managed by the kernel itself. The advantage? Here's an easy one: the transition from the splash screen to the display manager becomes flicker-free :-)

Basically, it means that as soon as the kernel gets hold of the display, it sets it to the highest settings it supports, immediately, and then holds on to it. So, during the transition from boot to X11 or vice versa, or any app that does not require a change in resolution, the earlier ever-present flicker is gone. Switching users, booting and shutting down the OS, as well as playing games that work at maximum settings, are a part of this package. If you talk about aesthetics... well, this is the king of it!

There's another advantage. Since the kernel itself manages the video memory instead of the X11 video driver, in case a Kernel Oops or panic occurs, the kernel can wipe out the X11 display and then show an error message. Compare this with the pre-KMS era where it would simply freeze up the system requiring a manual reset.

And here's another subtle aesthetic touch (the kernel developers just couldn't resist, could they?): the message can be decorated with full-screen, full-colour fancy images.

Well, as for the Fedora developers—KMS alone was just not good enough. They decided to enhance it.

Bootsplash was never used in Fedora and it had a few issues. It depended on the use of the framebuffer mode, which a number of systems didn't support well. Besides, it also required a kernel patch, including an in-kernel JPEG decoder, which was never accepted in the upstream kernel for obvious reasons.

Thus, prior to release 9, all versions of Fedora used an X11-based bootsplash program called RHGB, which as a program was quite heavy on resources. It required initialising the X server twice and de-initialising it in-between—initing it when RHGB started, de-initing it when RHGB ended, and then again initing it when the display manager started. This involved a lot of overhead and stressed the monitor.

In Fedora 9, this was done away with by a new system called Plymouth, which doesn't require X11. It can work in the old VESA framebuffer mode—that is, by adding a *vga=XfZ* parameter to the kernel (in GRUB) while booting—as well as take advantage of KMS. This is what it was designed for and this is where it excels.

It takes boot animations to a whole new level, showing transition effects, erupting solar flares and everything in between. And of course, you can always press a key to see the verbose boot messages. It is modular, event driven and all things modern. And, when KMS is available, the boot splash smoothly fades in to the display manager.

Plymouth has already tasted mainstream success, being included with the current Mandriva release. It is slated to replace Ubuntu's U Splash, and with all the inherent problems of Bootsplash and the fact that Plymouth can work with both UMS and KMS, we should be able to see most, if not all distros switch to Plymouth to provide a graphical boot.

And Upstart

Well, we've discussed the pretty splash screens that hide all the essential stuff that goes on in the background. When the system starts, all that switching of run-levels and starting/stopping services has to be managed by some software. A command called *init* was defined by the UNIX systems architecture to manage these. *init* is a privileged process, always executing with the PID 1. And if *init* dies, the kernel panics and the system reboots.

Up till now, the mainstream implementation of *init* was SysVinit, compatible with the *init* of UNIX System V. But SysVinit has a lot of problems. It's monolithic and synchronous, and does not support simultaneous operations. Obviously, the biggest hit in this implementation was speed.

SysVinit scripts are executable Bourne Shell scripts, and take three arguments during execution: start, stop and restart. Each script does its own housekeeping, manages its own PID and basically does a lot of redundant work. And these, while being simple to write syntactically, are difficult to comprehend.

Continued on page 45